

ReceiptMatcher AI

The Comprehensive Antigravity & Virality AI Implementation Playbook

1. Executive Summary & Main Goal

ReceiptMatcher AI is designed as a hyper-focused, low-overhead micro-SaaS created specifically for solopreneurs, freelance contractors, and micro-business owners. Standard AI chatbots are great for chatting, but they cannot connect securely with local financial structures. ReceiptMatcher bridges this gap by acting as a direct, camera-driven receipt reconciliation engine.

The Target Output Goal

\$3,000 – \$8,000 MRR

Achievable by securing 200–400 active monthly subscribers paying \$14.99/mo within 6 to 12 months.

Our Unique Advantage

1-Click Convenience

We bypass the friction of prompting. Snap a photo, auto-extract the data, and match to live database ledgers instantly.

2. Google Antigravity Agent Prompts

As a vibe coder, you don't need to write every line of backend architecture. Copy and paste the prompt configuration below directly into your Google Antigravity workspace to let the multi-agent system scaffold, verify, and execute the workspace setup.



Primary System Command for Google Antigravity

Build a web-based micro-SaaS using Python (FastAPI backend) and Streamlit (frontend) called "ReceiptMatcher".

The application must let users upload an image or take a photo of a receipt, parse key transaction details, and return them in a structured JSON format.

Organize the multi-agent workflow inside the workspace to execute concurrently:

- Agent 1: Initialize the local python virtual environment, configure environment variables for GEMINI API key.
- Agent 2: Define strict structured data models using Pydantic in 'schemas.py'. Construct parsing prompts for Gemini.
- Agent 3: Build a responsive, mobile-first Streamlit UI 'app.py' displaying a live camera stream triggered by a button click.
- Agent 4: Execute local servers, verify cross-origin API communication, and ensure the pipeline captures all data.

3. Step-by-Step Implementation Process

Your Antigravity agent system will scaffold the file structure. Below is the precise implementation code they should output for your verification.

A. Dependency Structure (requirements.txt)

```
fastapi==0.111.0
streamlit==1.35.0
google-generativeai==0.5.4
pydantic==2.7.1
uvicorn==0.30.1
requests==2.32.2
```

B. Structured Data Model (schemas.py)

Guarantees that the Gemini model returns a strictly formatted JSON object every single time.

```
from pydantic import BaseModel

class ReceiptData(BaseModel):
    vendor: str
    date: str
    amount: float
    tax: float
    category: str
```

C. Extraction Pipeline Engine (main.py)

Processes raw bytes over FastAPI using Gemini's flash-vision model.

```

import os
import json
from fastapi import FastAPI, UploadFile, File
import google.generativeai as genai
from schemas import ReceiptData

app = FastAPI()

genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))

SYSTEM_PROMPT = "You are a precision bookkeeping scanner. Analyze the receipt image. Extract the vendor, d

@app.post("/api/scan")
async def scan_receipt(file: UploadFile = File(...)):
    bytes_data = await file.read()
    image_parts = [{"mime_type": file.content_type, "data": bytes_data}]

    model = genai.GenerativeModel('gemini-1.5-flash')
    response = model.generate_content([SYSTEM_PROMPT, image_parts[0]])

    extracted_json = json.loads(response.text.strip())
    return extracted_json

```

D. Streamlit Mobile Interface (app.py)

Rendered with dynamic forms optimized for mobile phones and camera captures.

```

import streamlit as st
import requests

st.set_page_config(page_title="ReceiptMatcher AI", layout="centered")

st.title("📄 ReceiptMatcher AI")
st.caption("Auto-match physical receipts to your transaction ledger.")

uploaded_file = st.camera_input("Scan your paper receipt")

if uploaded_file is not None:
    st.info("Parsing invoice structure via Vision API...")

    files = {"file": (uploaded_file.name, uploaded_file.getvalue(), uploaded_file.type)}
    response = requests.post("http://localhost:8000/api/scan", files=files)

    if response.status_code == 200:
        data = response.json()
        st.success("Analysis Complete!")

        col1, col2 = st.columns(2)
        with col1:
            st.metric("Vendor Name", data['vendor'])
            st.metric("Total Bill", f"${data['amount']:.2f}")
        with col2:
            st.metric("Expense Date", data['date'])
            st.metric("Category", data['category'])

        st.markdown("### Match Ledger Transaction")
        st.radio("Select transaction to reconcile:", [
            f"Pending Match - {data['date']} - Wire Transfer - ${data['amount']:.2f}",
            "Add as manual ledger item"
        ])

        if st.button("Complete Reconciliation", type="primary"):
            st.balloons()
            st.success("Reconciliation successfully logged!")

```

4. Organic Distribution & Virality AI Playbook

Because we are ignoring AdMob/AdSense banners, your marketing relies on organic exposure. We utilize **Virality AI** to source hooks and algorithms without paying for ad placements.

The Virality AI Strategy: Use the analyzer to scrape trending sounds and video frameworks under keywords: #freelancerlife , #solopreneur , #taxseason , and #bookkeeperprobs . Replicate high-retention structural setups instead of guessing what works.

● The "Chaos to Order" Hook (TikTok/Reels)

Video setup: 7 seconds total. Show a giant, messy pile of receipts spread across your workspace desk.

Hook Overlay: *"Tax season is coming and you are still typing these by hand?"*

The Action: Snap a quick photo using ReceiptMatcher AI in real-time, showing it instantly convert into a structured row on a ledger database. Fast transitions drive immediate interest.

Reddit Forum Seeding (Stealth Marketing)

Avoid sales pitches. Participate as a builder sharing progress. Target specific online boards where stress peaks.

- **Target Boards:** `r/freelance` , `r/selfemployed` , `r/tax` , `r/bookkeeping` , `r/SideProject`
- **The Stealth Formula:** *"I spent hours typing paper bills every Sunday, so I wrote a short backend script that auto-extracts invoice items using Gemini and logs them onto my local CSV spreadsheets. I wrapped a basic web page around it so it's simple to use on my phone. Let me know if anyone wants to run their weekly stack through it to clear their backlog!"*

5. Monetization Strategy

Value-Based SaaS Pricing Matrix

Do not degrade your utility app with distracting, spammy AdMob overlays. Business owners and freelancers demand clean, secure software and will happily pay for direct utility.

- **Free Trial:** 5 receipt analysis scans per user account. Allows users to test speed and parsing accuracy with zero commitment.
- **Weekly Pass (\$2.99/week):** Ideal for freelance contractors who only clean up their ledgers once a month during crunch-time.
- **Premium Subscription (\$14.00/month):** Unlocks infinite image uploads, database matching ledgers, and direct multi-format CSV/Excel expense exporting.